Inter-Office Memo Management System

Requirements Specification

Course Code:	CSE226
Course Title:	Foundations of Vibe Coding
Department:	Electrical and Computer Engineering
Institution:	North South University
Semester:	Summer 2026
Submission Deadline:	Midnight, 29 August 2026

1. Project Overview

The objective of this project is to design and implement a web-based **Inter-Office Memo Management System** for managing internal organizational communications, approvals, reviews, and comments.

The system shall be a **multi-tenant application**, allowing multiple independent organizations to use the same deployed system while maintaining strict isolation of their users, data, documents, workflows, and activities.

A user should be able to create an office memo and define a sequence of users through whom the memo must pass. Each participant in the sequence may be required to review, comment on, approve, reject, or request changes to the memo. The system should maintain the complete history of the memo and its workflow.

The application should be implemented as a complete, functional, deployed web application. Students are free to choose the programming language, framework, database, hosting platform, AI coding tools, and other technologies used to implement the system.

2. Functional Requirements

2.1. Multi-Tenant Organization Management

The system shall support multiple independent organizations (tenants).

Each organization shall have:

- Organization name
- Organization identifier
- Logo or profile image
- Contact information
- Departments
- Users
- Organization-specific configuration

Data belonging to one organization must never be visible or accessible to users belonging to another organization.

The system must enforce tenant isolation at the application and data-access levels.

An organization administrator shall be able to:

- Create and manage departments
- Add or invite users
- Activate or deactivate users
- Assign users to departments
- Assign appropriate roles to users
- Update organization information

2.2. User Authentication

The system shall provide secure authentication.

Users shall be able to:

- Log in
- Log out
- Change their password
- Reset a forgotten password
- View and update their profile

A user profile shall contain, at minimum:

- Name
- Email address
- Designation
- Department
- Role
- Account status

The system should maintain secure authentication sessions and prevent unauthorized access to protected resources.

2.3. User Roles and Permissions

At minimum, the system shall support the following roles.

Organization Administrator

An organization administrator can:

- Manage organization information
- Manage users
- Manage departments
- Activate or deactivate users
- View organization-level statistics
- View organization-level memo information
- Manage memo categories
- Manage workflow templates

Regular User

A regular user can:

- Create memos
- Save drafts
- Submit memos
- View memos they are authorized to access
- Participate in assigned workflows
- Comment on assigned memos
- Approve or reject memos when authorized
- View their memo history
- Manage their own profile

Authorization must be enforced on the server side. Hiding a UI element must not be considered sufficient authorization.

3. Memo Management

3.1. Memo Creation

Users shall be able to create a memo containing:

- Automatically generated memo/reference number
- Subject/title
- Memo body
- Author
- Department
- Category/type
- Priority
- Creation date and time
- Attachments
- Workflow participants

The memo body should support basic rich-text formatting.
The system shall support the following priorities:

- Normal
- High
- Urgent

The system shall allow organizations to define memo categories, such as Administrative, Financial, Procurement, HR, Academic, Technical, and General.

3.2. Draft Memos

Users shall be able to save a memo as a draft.
Drafts shall be editable by their author.
A draft shall not enter the approval/review workflow until explicitly submitted by the author.
Users shall be able to:

- Create drafts
- Edit drafts
- Delete drafts
- Submit drafts

Once submitted, the system shall record the submission event.

4. Memo Workflow

The memo workflow is the central functionality of the system.
When submitting a memo, the author shall be able to define an ordered sequence of users who need to participate in the workflow.
For example:

$$\text{Employee} \rightarrow \text{Department Head} \rightarrow \text{Finance Manager} \rightarrow \text{Director} \rightarrow \text{CEO}$$

Each workflow participant shall have a specific position in the sequence.
The workflow shall proceed sequentially.
For example:

1. The employee submits the memo.
2. The Department Head receives the memo.
3. The Department Head approves it.

4. The memo moves automatically to the Finance Manager.
5. The Finance Manager approves it.
6. The memo moves to the Director.
7. The Director requests changes.
8. The author receives the request.
9. The author revises and resubmits the memo.
10. The workflow continues.

Only the user whose turn it currently is should normally be able to perform the corresponding workflow action.

4.1. Workflow Actions

Depending on the workflow step, a participant shall be able to perform one or more of the following actions:

- Approve
- Reject
- Comment
- Request Changes
- Forward/Complete Review

The action performed shall be recorded permanently in the memo's history.
An approval shall identify:

- User
- Action
- Date and time
- Optional comment

A rejection shall require a reason or comment.
A request for changes shall require a comment explaining what needs to be changed.

4.2. Workflow Sequence

The system shall ensure that workflow steps are executed in the defined order.
For example, if a memo has the sequence:

$$A \rightarrow B \rightarrow C \rightarrow D$$

User C must not be able to approve the memo while the memo is still awaiting action from B.
The system shall clearly identify:

- Completed steps
- Current step
- Future steps
- User responsible for the current step
- Action taken at each completed step

4.3. Workflow Completion

When the final workflow participant approves the memo, the memo shall be marked as **Approved/Completed**.
The system shall record:

- Final approver

- Final approval date/time
- Complete workflow history

A completed memo should become read-only to ordinary users, except where an authorized administrative or versioning mechanism explicitly permits further action.

4.4. Rejection and Changes

A workflow participant may reject a memo or request changes.

The system shall distinguish between:

- **Reject:** The workflow terminates and the memo becomes rejected.
- **Request Changes:** The memo is returned to the appropriate previous participant, normally the author, for modification and resubmission.

The system shall preserve the history of previous submissions and decisions.

5. Memo Status

The system shall support at least the following statuses:

- Draft
- Submitted
- Pending Review
- Pending Approval
- Changes Requested
- Rejected
- Approved
- Cancelled

The current status shall be clearly visible to authorized users.

The current workflow participant and required action shall also be displayed.

6. Memo Inbox and Outbox

6.1. Inbox

Each user shall have an inbox containing memos requiring their action.

The inbox should display:

- Memo number
- Subject
- Sender
- Department
- Priority
- Current status
- Date submitted
- Required action
- Age/time pending

Users should be able to filter and sort the inbox.

6.2. Sent / My Memos

Users shall be able to view memos they have created or submitted.

The list shall display:

- Memo number
- Subject
- Status
- Current workflow participant
- Priority
- Submission date
- Last activity date

6.3. Completed Memos

Users shall be able to view completed workflows for memos they are authorized to access.

7. Memo Details and Timeline

The memo details page shall display the complete memo and its workflow information.
It shall contain:

- Memo number
- Subject
- Author
- Department
- Category
- Priority
- Body
- Attachments
- Current status
- Current workflow participant
- Workflow sequence
- Comments
- Activity history

The system shall display a chronological timeline.
For example:

10:32 AM — Memo created by A
10:35 AM — Memo submitted
11:14 AM — B approved
11:18 AM — Memo forwarded to C
12:03 PM — C requested changes
1:20 PM — A resubmitted memo

The timeline shall identify the user, action, timestamp, and relevant comment where applicable.

8. Comments and Discussion

Users participating in a memo workflow shall be able to add comments.
Each comment shall contain:

- Comment author
- Comment text
- Date and time

Comments shall be displayed chronologically.

Ordinary users should not be able to silently modify or delete historical workflow comments.

The system should clearly distinguish between:

- General comments
- Approval comments
- Rejection reasons
- Change requests

9. Attachments

Users shall be able to attach files to memos.

The system shall support:

- File upload
- File download
- File name display
- File size display
- Uploading user
- Upload timestamp

The system shall enforce reasonable file-size and file-type restrictions.

Attachment access shall follow the permissions of the associated memo.

A user must not be able to access an attachment merely by guessing or manipulating its URL.

10. Notifications

The system shall provide notifications for important workflow events.

Users should be notified when:

- A memo requires their action
- A memo is approved
- A memo is rejected
- Changes are requested
- A comment is added
- A memo is resubmitted
- A workflow is completed
- A user is assigned to a workflow

Notifications should be accessible through an in-application notification interface.

The system may additionally support email notifications.

Users should be able to identify unread notifications.

11. Search and Filtering

The system shall provide search functionality within the user's authorized organizational data.

Users should be able to search by:

- Memo number
- Subject
- Memo body
- Author
- Department
- Category

- Status
- Priority
- Date range

Search results must respect authorization and tenant boundaries.

A user must never receive search results from another organization or from a memo they are not authorized to access.

12. Dashboard

The system shall provide a dashboard for users.

The dashboard should display:

- Memos awaiting the user's action
- Memos submitted by the user
- Recently completed memos
- Pending approvals
- Pending reviews
- Urgent memos
- Recent activity
- Memo counts by status

The organization administrator dashboard should additionally provide organization-level information such as:

- Number of users
- Number of active users
- Number of departments
- Number of memos
- Pending workflows
- Completed workflows
- Rejected workflows
- Recent system activity

13. Department Management

Each organization shall be able to define departments.

A department shall have:

- Department name
- Description
- Status

Users shall be associated with a department.

Organization administrators shall be able to:

- Create departments
- Rename departments
- Deactivate departments
- Assign users to departments

Deactivating a department must not delete historical memo information.

14. Memo Categories

Organization administrators shall be able to create and manage memo categories.

Each category shall have:

- Name
- Description
- Active/inactive status

Categories may be used to organize and filter memos.

15. Workflow Templates

The system shall support reusable workflow templates.

An administrator shall be able to define templates such as:

Purchase Request

Employee → Department Head → Finance → Director

Leave Request

Employee → Line Manager → HR

Procurement Request

Requester → Department Head → Procurement → Finance → Director

A template should specify the ordered workflow positions.

When creating a memo, the author may select an appropriate workflow template and assign users to the corresponding positions.

The system should also allow a user with appropriate permissions to define a custom workflow for an individual memo.

16. Delegation

The system should support workflow delegation.

A user should be able to designate another authorized user to act on their behalf for a specified period.

A delegation should contain:

- Delegating user
- Delegate
- Start date
- End date
- Optional reason
- Status

Actions performed by a delegate must clearly identify both the delegate and the user on whose behalf the action was performed.

17. Memo Versioning

When a memo is returned for changes, the system should maintain versions.

Each version should record:

- Version number
- Author/editor

- Modification date/time
- Memo content
- Associated submission

Users with appropriate permissions should be able to view previous versions.
The system must not silently overwrite the historical version of a submitted memo.

18. Audit Log

The system shall maintain an audit log of significant system events.
The audit log should record events including:

- User login
- User logout
- User creation
- User activation/deactivation
- Memo creation
- Memo modification
- Memo submission
- Workflow assignment
- Comment
- Approval
- Rejection
- Change request
- Resubmission
- Attachment upload
- Attachment deletion
- Workflow completion

Each audit record should contain:

- Event type
- User
- Organization
- Date/time
- Relevant object/entity
- Description

Audit records should not be editable by ordinary users.

19. Reporting

The system should provide basic reporting functionality.
Administrators should be able to view statistics such as:

- Number of memos by status
- Number of memos by department
- Number of memos by category
- Number of urgent memos
- Average workflow completion time
- Number of pending approvals
- Number of rejected memos
- Number of change requests

Reports should support appropriate filtering by date range, department, category, and status.

20. PDF Export

The system should allow an authorized user to export a memo as a PDF.

The exported document should contain:

- Organization information
- Memo number
- Subject
- Author
- Department
- Date
- Memo body
- Attachments or attachment references
- Workflow participants
- Approval history
- Comments
- Final status

The PDF should clearly indicate whether the memo is approved, rejected, or still in progress.

21. Security Requirements

Security is an essential requirement of the system.

The implementation shall:

1. Authenticate all protected users and operations.
2. Authorize all protected operations on the server side.
3. Enforce strict tenant isolation.
4. Prevent users from accessing another organization's data.
5. Prevent users from accessing unauthorized memos.
6. Prevent unauthorized workflow actions.
7. Protect stored passwords using appropriate password hashing.
8. Protect authentication credentials and session information.
9. Validate user input.
10. Protect against common web vulnerabilities.
11. Validate uploaded files.
12. Prevent unauthorized access to uploaded attachments.
13. Avoid exposing sensitive information through error messages.
14. Use HTTPS for the deployed application.
15. Ensure that database queries are appropriately protected against injection attacks.

Tenant isolation shall be treated as a fundamental system requirement rather than merely a user-interface feature.

22. User Interface Requirements

The application shall provide a usable web interface.

At minimum, the interface shall include:

- Login page
- Dashboard
- Inbox
- My Memos
- Memo creation page

- Memo details page
- Workflow interface
- Notifications
- Search/filter interface
- User profile
- Administration interface

The interface should be responsive and usable on both desktop and mobile-sized screens.

The current workflow state and required user action should be visually obvious.

23. Deployment Requirements

The completed application must be deployed and accessible through a publicly reachable URL.

Students shall provide:

Deployed System URL: `[Insert URL]`

The deployed system must be functional at the time of submission.

Students should provide appropriate demonstration accounts if the system requires authentication.

If administrator functionality is part of the demonstration, an administrator demonstration account should also be provided.

Students are responsible for ensuring that the deployed application remains accessible for evaluation.

24. Source Code Submission

Students shall submit a link to a ZIP archive containing the complete source code.

Source Code ZIP URL: `[Insert URL]`

The ZIP archive shall contain:

- Complete source code
- Configuration files required to run the application
- Database schema/migrations
- Seed/demo data where applicable
- Dependency definitions
- Environment-variable template
- Installation instructions
- Build instructions
- Run instructions

The ZIP archive must contain everything necessary for another developer to understand and set up the system, except for secrets such as passwords, API keys, private keys, or production credentials.

25. Installation and Setup Documentation

The submitted source code must include clear installation instructions.

The instructions shall describe:

1. Required software and versions.
2. How to install dependencies.

3. How to configure environment variables.
4. How to configure the database.
5. How to initialize the database.
6. How to create or seed demonstration data.
7. How to start the application locally.
8. How to build the application for production, where applicable.
9. Any external services required by the application.
10. Any additional configuration required to reproduce the deployed system.

An evaluator should be able to follow the instructions and run the application without requiring undocumented steps.

26. Project Documentation Submission

In addition to the source code, students shall submit a document describing their implementation.

The document shall include:

26.1. System Overview

A brief description of the implemented system and its major functionality.

26.2. Requirements Implemented

Identify which requirements from this specification have been implemented.

26.3. Technology Stack

Describe:

- Programming languages
- Frontend framework
- Backend framework
- Database
- Authentication mechanism
- Hosting/deployment platform
- External services
- AI/vibe-coding tools used

26.4. System Architecture

Provide a high-level description and preferably a diagram showing:

- Frontend
- Backend
- Database
- Authentication
- File storage
- External services
- Major system components

26.5. Database Design

Describe the main entities and relationships.

The documentation should explain how multi-tenancy is implemented.

26.6. Workflow Design

Explain how the sequential memo approval/review workflow is represented and implemented.

26.7. Security

Describe how the system handles:

- Authentication
- Authorization
- Tenant isolation
- File security
- Password security
- Other relevant security considerations

26.8. Vibe-Coding Process

Students shall describe how AI-assisted/vibe coding was used during development.

This should include:

- AI tools used
- How requirements were communicated to the AI
- How generated code was evaluated
- How errors were identified and corrected
- How the students verified that the generated system actually satisfied the requirements

26.9. Known Limitations

Students should identify functionality that was not implemented, known bugs, limitations, or technical compromises.

26.10. Deployment Information

The document must include:

Live System: `[Insert deployed-system URL]`

Source Code ZIP: `[Insert source-code ZIP URL]`

Installation Instructions: Describe or provide the location of the installation instructions.

27. AI Prompt and Response History

Because this is a course on Vibe Coding, the use of AI during development must be documented transparently.

Students shall submit the **complete prompt and response history** of their AI-assisted development process.

This requirement applies to every AI coding assistant or AI system used substantially during the development of the project.

The submission shall contain, as applicable:

- Every significant prompt provided by the student to an AI system.
- The corresponding AI-generated responses.
- Prompts used to generate application code.

- Prompts used to modify or refactor code.
- Prompts used to debug errors.
- Prompts used to diagnose deployment problems.
- Prompts used to design or modify the database.
- Prompts used to design the user interface.
- Prompts used to implement or modify authentication and authorization.
- Prompts used to implement the memo workflow.
- Prompts used to test or validate functionality.
- Subsequent prompts used to correct or improve AI-generated code.

The history should preserve the chronological order of the interaction.

The submission should make it possible to understand how the system evolved through the interaction between the student and the AI system.

Where the AI tool provides a facility for exporting conversation history, students should submit the exported conversation directly rather than manually rewriting it.

If multiple AI tools were used, the history from each tool shall be included.

The prompt and response history shall not be replaced by a summary. A summary of AI usage may be included in the project documentation, but it does not satisfy the requirement to submit the actual interaction history.

Students should not intentionally omit relevant AI interactions merely because those interactions produced incorrect, incomplete, or unusable code. Debugging, failed approaches, corrections, and revisions are part of the development history and should be retained.

The prompt and response history may be submitted as one or more files, such as:

- PDF
- HTML
- Markdown
- Plain text
- Exported AI conversation format

The submitted history should be readable and clearly associated with the student or group submitting the project.

Students must not include passwords, API keys, private keys, access tokens, or other confidential credentials in the submitted prompt and response history. If such information was accidentally included in an AI interaction, the student should redact the secret while preserving the surrounding interaction and clearly indicating that a credential has been redacted.

27.1. AI History Submission Location

The project documentation shall contain a link to the complete AI prompt and response history.

AI Prompt/Response History URL: `[Insert URL]`

The linked material must remain accessible for evaluation.

28. Demonstration Requirements

The deployed system should be capable of demonstrating at least the following scenario:

1. Create an organization.
2. Create multiple users belonging to that organization.
3. Create a memo.
4. Define a sequential workflow involving multiple users.
5. Submit the memo.

6. Log in as the first workflow participant.
7. Comment, approve, reject, or request changes.
8. Demonstrate the memo moving to the next participant.
9. Demonstrate the complete workflow history.
10. Demonstrate final approval or rejection.
11. Demonstrate notifications.
12. Demonstrate search and filtering.
13. Demonstrate administrative functionality.
14. Demonstrate that a user from another organization cannot access the memo.

The evaluator may use the deployed application to test functionality beyond the demonstration scenario.

29. Submission Requirements

Each student or group shall submit the following before the deadline.

A. Deployed Application

A publicly accessible URL to the completed system.

URL: `[Insert URL]`

B. Project Documentation

A document describing:

- What was implemented
- How it was implemented
- Technology stack
- Architecture
- Database design
- Workflow design
- Security considerations
- Vibe-coding/AI-assisted development process
- Known limitations

C. Source Code

A link to a ZIP file containing:

- Complete source code
- Database schema/migrations
- Configuration files
- Dependency definitions
- Installation instructions
- Setup instructions
- Required supporting files

ZIP URL: `[Insert URL]`

D. AI Prompt and Response History

A link containing the complete prompt and response history from the AI systems used during development.

AI History URL: `[Insert URL]`

E. Demonstration Credentials

Where authentication is required, students shall provide suitable demonstration credentials for evaluating the application.

Production secrets must never be included in the submission document, source code archive, or AI prompt/response history.

30. General Technical Expectations

Students are free to select their own technology stack.

There is no prescribed requirement regarding:

- Programming language
- Frontend framework
- Backend framework
- Database technology
- Cloud provider
- AI coding assistant
- Development environment

However, the final system must be functional, maintainable, reasonably secure, and demonstrably compliant with the requirements in this specification.

The use of AI coding assistants is expected as part of the course. However, students remain responsible for understanding, testing, debugging, integrating, and validating the code produced with AI assistance.

A system that merely presents a convincing user interface but does not correctly implement the underlying functionality, data persistence, authorization, workflow, or tenant isolation will not satisfy the requirements.

The submitted AI interaction history may be used to evaluate the student's development process, including how requirements were communicated, how AI generated solutions were evaluated, and how problems were identified and resolved.

31. Submission Deadline

Midnight, 29 August 2026

All required submission links and documentation must be submitted by the stated deadline.

The deployed application must be accessible for evaluation, the submitted source-code archive must be complete and accompanied by sufficient installation instructions, and the complete AI prompt and response history must be accessible through the submitted link.